

第4关-精华笔记

课程目标

1. 熟练掌握列表、字典中元素的增删改查
2. 理解列表和字典的区别

课程难点

1. 列表与字典增删改查的异同
2. 正确使用切片，深刻理解切片时冒号左右数字的意义

课程重要内容

一、列表

语法格式

数据存储在**中括号[]**里，用逗号隔开并使用等号赋值给列表。中括号里面的每一个数据叫作“元素”。

列表中的元素是有自己明确的“位置”的，元素相同，在列表中排列顺序不同，就是两个不同的列表。

列表中字符串、整数、浮点数都可以存储。

```
1 list1 = ['小明',17,5.2]
```

提取元素

偏移量：元素在列表中的编号。

偏移量是从0开始的；

列表名后加带偏移量的中括号，就能取到相应位置的元素。

结果是一个元素

切片：用冒号来截取列表元素的操作。

冒号左边空（或者为0），：m，表示从头取m个元素；

右边空（或者为0），n：，跳过前n个元素把剩下的取完；

冒号左右都有数字时，n：m，表示跳过前n个元素，取到第m个。（取出前m个元素中除了前n个后剩下的那些）

切片截取了列表的一部分，所以得到的结果仍然是一个列表。（即使只有一个元素，也是在列表里的，要与用偏移量取单个元素的方法区别开）

```
1 students = ['小明','小红','小刚']
2 print(students[2])
3 #使用偏移量提取单一元素，结果显示：
4 #小刚
5 print(students[2:])
6 #使用切片，即使结果仍然只有一个元素，但显示为列表：
7 #['小刚']
```

特别地，a,b,c=students，也可以提取出列表中的元素，变量分别用逗号隔开，且变量的数量与列表元素个数一致，一一对应，最终列表元素会分别赋值给变量，例如：

```
1 appetizer = ['话梅花生','拍黄瓜','凉拌三丝']
2 a,b,c = appetizer
3 print(a)
4 print(b)
5 print(c)
6 print(a,b,c)
7 #结果显示为
8 #话梅花生
9 #拍黄瓜
10 #凉拌三丝
11 #话梅花生 拍黄瓜 凉拌三丝
```

增加/删除元素

增加元素

列表名.append()

注意这里是.不是空格！append后的括号里只能接受一个参数，结果是让列表末尾新增一个元素。列表长度可变，理论容量无限，所以支持任意的嵌套。

```
1 list3 = [1, 2]
2 #添加‘3’这个元素
3 list3.append(3)
4 print(list3)
5 #结果显示：
6 #[1,2,3]
7 list3.append(4, 5)
8 list3.append([4, 5])
```

```

9 print(list3)
10 #添加‘[4, 5]’这个列表，也就是append()的参数为一个列表，也是一个参数，所以不会报错
11 #结果显示：
12 #[1, 2, 3, [4, 5]]

```

但是append(4,5)会报错，因为给了两个元素（没有作为一个整体，所以算两个参数）。

注意！！ 千万不能赋值：a=student.append(3)，这样a里只有none。

删除元素

del 列表名[元素的索引] 。

注意这里是空格不是！ 因为del是语句，是命令。

与append()函数类似，能删除单个元素、多个元素（切片）、整个列表。

修改元素

使用偏移量修改对应位置的元素。

```

1 list1 = ['小明','小红','小刚','小美']
2 list1[1]='小蓝'
3 print(list1)
4 #结果显示
5 #['小明','小蓝','小刚','小美']

```

二、字典

字典所存储的两种数据若存在一一对应的情况，用字典储存会更加方便。唯一的键和对应的值形成的整体，我们就叫做【键值对】。键具备唯一性，而值可重复。

语法格式

- 字典外层是大括号{}，用等号赋值；
- 列表中的元素是自成一体的，而字典的元素是由键值对构成的，用英文冒号连接。有多少个键值对就有多少个元素。如'小明':95，其中我们把'小明'叫键（key），95叫值(value)。
- 键值对间用逗号隔开
- 字典中数据是随机排列的，调动顺序也不影响。所以列表有序，要用偏移量定位；字典无序，便通过唯一的键来定位。（注：len()函数用于获取数据长度）

```

1 students = ['小明','小红','小刚']
2 scores = {'小明':95,'小红':90,'小刚':100}
3 print(len(students))
4 print(len(scores))
5 #结果显示
6 #3
7 #3
8 #字典的元素个数，数冒号就行了

```

提取元素

字典没有偏移量，所以在提取元素时，中括号中应该写键的名称，即字典名[字典的键]。提取出来的是key对应的value，而不会显示键的数据！

```

1 scores = {'小明': 95, '小红': 90, '小刚': 90}
2 print(scores['小明'])
3 #结果显示
4 #95

```

增加/删除元素

新增元素

字典名[键] = 值。

每次只能新增一个键值对。scores['小刚','小美']=92,85，这样是不对的，最终会输出('小刚','小美):(92,85))作为一个键值对。

```

1 album = {'周杰伦':'七里香'}
2 album['林俊杰'] = '小酒窝'
3 print(album)
4 print(album['周杰伦'])
5 #结果显示
6 #{'周杰伦':'七里香', '林俊杰':'小酒窝'}
7 #七里香

```

删除元素

del 字典名[键]

```

1 album = {'周杰伦':'七里香','王力宏':'心中的日月'}
2 del album['周杰伦']
3 print(album)
4 #结果显示
5 #{'王力宏':'心中的日月'}

```

修改元素

如果不是整个键值对都不需要，只需要改变对应key下的value，修改就可以，不需要del。

```
1 dict1 = {'小明':'男'}
2 print(dict1)
3 #字典没有偏移量，只能通过key找到元素位置
```

三、列表与字典的嵌套

列表嵌套列表

列表中有两个列表元素，[1]表示取第二个元素（列表），[2]表示取第二个元素中的第三个元素（偏移量为2）。

```
1 student=[['小红','小黄','小橙'],['小绿','小蓝','小紫','小青']]
2 print(student[1][2])
3 #结果显示为
4 #小紫
```

字典嵌套字典

字典中存储了两个字典，所以提取数据时只能用key值。

```
1 scores={'第一组':{'小明':95,'小红':96},'第二组':{'小刚':94,'小青':99}}
2 print(scores['第一组']['小红'])
3 #结果显示:
4 #96
```

列表嵌套字典

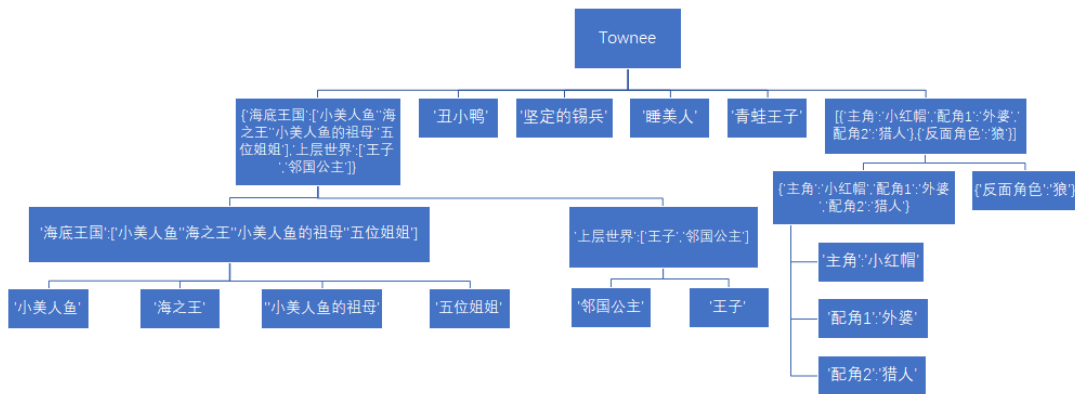
使用偏移量从最外层括号到最内层括号取数。查找townee列表中的第六个元素中的第2个元素（一定是字典，因为之后用的是键值而不是偏移量）中key为‘反面角色’的value。

```
1 townee = [
```

```

2     {'海底王国':['小美人鱼','海之王','小美人鱼的祖母','五位姐姐'],'上层世界':['王
子','邻国公主']},
3     '丑小鸭','坚定的锡兵','睡美人','青蛙王子',
4     [{'主角':'小红帽','配角1':'外婆','配角2':'猎人'},{'反面角色':'狼'}]
5 ]
6 print(townee[5][1]['反面角色'])

```



列表里面的嵌套，第一个元素是个字典，以邻国公主后的那个逗号结尾的部分是列表的第0偏移量，第1个偏移量是丑小鸭，第2个是锡兵，第3个偏移量是睡美人，第4个偏移量是青蛙王子，狼是第5个偏移量里面的列表的第一个偏移量，通过字典的键取值就可以取出来了

```

1 townee = [
2     {'海底王国':['小美人鱼','海之王','小美人鱼的祖母','五位姐姐'],'上层世界':['王
0 子','邻国公主']},2
3     '丑小鸭',3
4     '坚定的锡兵',4
5     '睡美人',
6     '青蛙王子',
7     [{'主角':'小红帽','配角1':'外婆','配角2':'猎人'},{'反面角色':'狼'}]
8 ]
9
10    5 0 1
11    print(townee[5][1]['反面角色'])

```

- 1、townee是个列表，有6个值，狼在第6个里边，先得出townee[5]
- 2、townee[5]也是一个列表，有两个值，每个值是一个字典，狼在第2个字典里，得出townee[5][1]
- 3、因为townee[5][1]是个字典，反面角色的值是狼，所以可以得出townee[5][1]['反面角色']